



Security Controls in Shared Source Code Repositories

CSD380: DevOps
Module 10.2 Assignment
Isaac Ellingson
7/26/2026



Securing Your Source Code Repository

- Protect your Main Branch
- Require GPG Signatures for Commits
- Use Secret Management Tools
- Review Repository Activity



Protect your Main Branch

- All other branches depend on monotonic behavior in main – where history remains the way it was, and only moves forward
- Force-pushes to main can rewrite history and create unsolvable merge conflicts
- Protecting a branch disables force-push and branch-delete automatically on GitHub
- Branch protection also gives you granular settings to require reviews, ensure passing tests, and control access.



Protect your Main Branch

- On GitHub, from your repository page, click “Settings” up top with the gear icon next to it.
- Click “Branches” in the left column of Settings pages
- Click “Add Branch Ruleset”
- Enter something like “main” in Ruleset name
- Under Branch Targeting Criteria, click “Add target” → Include Default Branch
- Click “Create” at the bottom of the page.



Require GPG Signatures for Commits

- Unsigned commits can allow malicious edits to code without a clear way to discover who did them
- Unsigned commits can allow careless edits without any accountability
- Commit signatures encourage a culture of responsibility among devs
- Having someone logged gives you a qualified, invested person to help solve any problems with the code
- Remember what we learned about blameless audits when looking at “git blame”!



Require GPG Signatures for Commits

- On GitHub, go back into your Branch Protection rule
- You can find existing rules under Settings > Rules > Rulesets
- Check “Require signed commits”
- Click “Save Changes”
- That’s it!



Use Secret Management Tools

- Committing things like API keys, secret keys, passwords, and server configs to your repository WILL expose your secrets
- Most software forges have an escrow system that can provide secrets and hidden system variables.
- Using these tools greatly improves the security for solutions that need to deploy ready-to-use with CI / CD.



Use Secret Management Tools

- On GitHub, from your repository page, go to Settings > Secrets and variables > Actions
- If your secret is used by a GitHub Action, use “Repository Secrets” to add a secret
- If your secret is used by Maven or Gradle, use “Environment Secrets” to add a secret
- More info:
<https://docs.github.com/en/actions/how-tos/write-workflows/choose-what-workflows-do/use-secrets>



Review Repository Activity

- As we have learned earlier in this course, unreviewed logs provide no security.
- The longer it takes to detect unwanted behavior, the harder it can be to correct it.
- Reviewing git activity dovetails nicely with code review
- Reviews can not only improve repository security, but also hygiene and code quality!



Review Repository Activity

- On GitHub, on the top right corner of the web UI, you have an “inbox” with updates on repositories, issues, pull requests, etc. that you are involved with.
- Click the “Done” checkbox when you’ve looked into the event.
- That’s it. You reviewed the activity!



Source Code Repositories are a Team Effort

Everyone participates in source control. That makes everyone a part of the security environment. You don't have to do everything yourself. (in fact, you can't!)



Thank You!

References

About protected branches. (2026). GitHub Docs.

<https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches>

Harness. (2025, June 26). Best practices for securing code repositories. Harness.io; harness.io.

<https://www.harness.io/harness-devops-academy/secure-code-repositories-best-practices>

Using secrets in GitHub actions - GitHub docs. (2025). GitHub Docs.

<https://docs.github.com/en/actions/how-tos/write-workflows/choose-what-workflows-do/use-secrets>

